

Ephemeral Sandbox: A LayerStack Workspace OS for Parallel Coding Agents

Provisional working title

[AUTHOR INFORMATION NEEDED]
[AFFILIATION INFORMATION NEEDED]

Draft manuscript scaffold

Abstract

Provisional design-first draft—not submission text. Agent systems can fan out intelligence, but conventional operating-system workspaces provide no agent-level protocol for fanning concurrent writes back into one durable project. Coding-agent teams and swarms inspect changing code, perform multi-file edits, run tests, and start services; sharing one mutable workspace permits live-state interference, while copying workspaces alone defers integration. We present Ephemeral Sandbox, a workspace OS organized around LayerStack: one durable project history, many private executable sessions, and controlled publication. A sessionless `exec_command` tool call receives an implicit isolated session over a leased LayerStack snapshot. When its command ledger drains, the runtime captures the private overlay, reconciles the complete delta against the current head, and either publishes one new layer or returns a structured rejection. Eligible text divergence can use a bounded three-way merge; structural, binary, oversized, or conflicting changes reject. Explicit sessions let multiple command and file tool calls share private state before publication or discard. Source and correctness tests establish these mechanisms, but not formal security, semantic merge correctness, process-state rollback, resource coordination, or improved multi-agent throughput. Whether Ephemeral Sandbox raises the workload-dependent useful-work concurrency ceiling remains an empirical question.

1 Introduction

Draft.

2 Goals, Non-goals, and Threat-model Boundary

The first goal of Ephemeral Sandbox is stable private execution over changing shared history. A workspace session should continue to observe the manifest and ordered layer set acquired by its lease even as later publications advance the shared head. Its shared lower layers should remain read-only from the session’s perspective, while its filesystem writes accumulate in a session-unique private overlay. At the end of that private phase, capture should faithfully translate the relevant OverlayFS state—including writes, deletions, links, and directory metadata—into typed changes or explicit protected drops. This is a filesystem-change contract, not process checkpointing.

The second goal is controlled integration with explicit outcomes. A captured candidate changeset is reconciled against the current active head using the implemented path-fingerprint, structural, protected-path, and bounded line-oriented text-merge rules. Every proposed data change must resolve, or the candidate is rejected as a whole. On acceptance, the runtime provides *atomic data publication*: one fully resolved data changeset becomes visible through the durable active-manifest transition after its layer is promoted. Attribution, accounting, and session cleanup occur outside this atomic data boundary. The lifecycle therefore distinguishes rejection or precommit failure, which can leave an explicit session available for retry, from successful or no-op publication and from the published-but-not-closed outcome caused by later cleanup failure.

A third goal is a role-separated operational contract for orchestration. Management, sandbox-scoped runtime work, and read-only observability are exposed through distinct client surfaces, with typed scope, request correlation, machine-readable results, and lifecycle outcomes. These surfaces let an external orchestrator invoke and inspect narrower classes of operation; their separation is an interface and least-exposure property, not by itself an authorization or security boundary.

The design also makes isolation, publication, scaling, and useful-work behavior measurable. Evaluation must test private-view and publication correctness, partial failures, resource and latency scaling, and verification-passing durably accepted work under stated workloads and runtime configurations. These are evaluation goals rather than achieved results: the audited baseline does not establish a performance advantage, a resource advantage, improved agent productivity, or a higher workload-dependent concurrency ceiling.

We use *workspace OS* in a deliberately bounded sense. It denotes the runtime substrate that supplies LayerStack history, workspace-session execution, filesystem capture and publication, lifecycle handling, and operational observability for coding-agent work. It does not denote a general-purpose operating system, kernel, hypervisor, security monitor, or complete coordination plane.

The threat model is correspondingly narrow. The mechanism addresses accidental interference through session filesystem state and stale or overlapping publication under the audited runtime contract. It does not provide a formal security proof, complete noninterference, sandbox-escape resistance, or universal network-egress denial; an implicit command session currently uses a shared-network profile, and isolated networking is only a selectable profile. It captures neither process state nor a process rollback point, and a clean textual reconciliation does not guarantee semantic compatibility, a successful build, or passing tests.

Finally, Ephemeral Sandbox does not automatically decompose tasks, arrange inter-agent communication, schedule workers, negotiate intent, transfer ownership, or implement a general agent-coordination plane. The audited mechanism is centered on Linux namespaces and OverlayFS and makes no universal cross-platform support or durability claim. Workspace isolation can remove one source of interference, but it cannot by itself solve collaboration, selection, integration policy, or verification. With these goals and boundaries fixed, the next section defines the state and invariants to which the mechanism is held.

3 System Model and Invariants

A *project history* is the runtime-recorded filesystem history from which workspace views are projected. Its *LayerStack* is the ordered, manifest-selected base plus promoted published or squash layers. A *layer* is one such filesystem contribution; promoted layers are content-digested and treated as append-oriented and immutable by the runtime, without claiming filesystem-enforced immutability against external mutation. A versioned *manifest* records the ordered layer references, and its derived root hash identifies that logical view. The *active head* is the currently shared durable project view selected by the active manifest. A *lease* captures one exact manifest and its resolved ordered layer-path set, retaining those layers or a safe substitute until release.

A *workspace session* is a runtime-owned private executable workspace over a leased LayerStack view. Its *private overlay* combines shared read-only lower layers with a session-unique writable upper/work pair. An *implicit session* is created only for a sessionless `exec_command`; after its command ledger drains, its policy is to publish and then destroy the session. An *explicit session* is created by a caller and can preserve one live private overlay across multiple command and file operations until deliberate publication or destruction. Sessionless file operations follow different paths: reads project the active LayerStack, whereas writes and edits amend the current head directly. The model therefore does not claim that every tool call creates a workspace session.

Capture walks a private upper directory and interprets OverlayFS metadata to produce typed filesystem changes and protected drops. Their complete proposed delta is the *candidate changeset*; it is not yet an accepted patch, a durable layer, or evidence of semantic validity. *Current-head reconciliation* rereads the active manifest, compares candidate paths with leased-base and current-head fingerprints, applies only the eligible bounded text-merge rules, and either resolves every change or rejects the data changeset. *Publication* comprises planning, reconciliation, staging, synchronization, layer promotion, and the active-manifest transition. *Rejection* is a publication outcome that leaves the active manifest unchanged. Other outcomes

distinguish precommit failure, no-op publication, successful publication and closure, and publication followed by a cleanup failure; the last is reported as published-but-not-closed rather than rolled back.

The state categories are orthogonal. *Shared state* includes the runtime’s recorded base, promoted layers, manifest-selected history, and active head; lease bookkeeping is shared runtime metadata, while the view captured by a lease belongs to its session. *Private state* includes a session’s upper/work directories, uncommitted filesystem changes, and live command and process state. *Durable state* at successful data publication is the promoted layer and digest together with the active manifest that selects it. *Publicly visible operational state* means state exposed to other runtime callers through the active LayerStack, lifecycle results, or read-only operational views; it does not mean public Internet visibility and is not synonymous with durability. The accepted filesystem data becomes publicly visible at the active-manifest transition. Best-effort attribution, accounting, observability records, and later cleanup are outside that atomic data boundary and can have different persistence and failure behavior.

An *orchestrator* is an external controller that selects work, invokes role-appropriate operations, and interprets outcomes; a *worker* performs one or more coding tasks through those operations. These roles do not imply that the runtime itself decomposes or schedules work. For evaluation, an *agent team* is the workload family with relatively stable roles, delegated tasks, dependencies, handoffs, and an orchestration or integration lane. An *exploratory swarm* is the workload family with redundant or competing work, weaker central coordination, hot-path overlap, rapidly changing bases, and result selection. These are workload families rather than universal product taxonomies. For workload W , runtime R , agent configuration A , and worker count n , *useful work* is verification-passing, durably accepted contribution units per unit wall time or resource cost. The *concurrency ceiling* is the workload- and runtime-specific region in which adding workers no longer improves that rate because conflict, retry, integration, verification, selection, or resource costs dominate marginal accepted progress. Both the shape of this region and whether Ephemeral Sandbox changes it remain hypotheses for evaluation.

Core invariants. The model requires the following four evidence-bounded properties.

1. A workspace session observes the manifest snapshot and ordered layer set captured by its lease at creation. Compaction may replace a physical representation only when a safe substitute preserves that leased logical view.
2. Filesystem writes made through a workspace session remain in its private overlay until successful publication. Capture creates a candidate changeset but does not itself expose the changes through the active head.
3. Publication either makes one fully resolved data changeset visible through a durable active-manifest transition or leaves the active manifest unchanged. This all-or-none property excludes attribution, accounting, and cleanup; a failure in those later phases does not undo an already visible data publication.
4. Compaction or garbage collection must not reclaim layers still pinned by a live lease unless a safe substitute preserves the leased logical view. The baseline source and lease-aware tests establish the in-process mechanism, but the lease/substitution registries are in memory and their behavior across daemon restart still requires fault testing.

These invariants specify a leased filesystem view and an atomic data-publication boundary. They do not constitute a full transaction, serializable snapshot isolation, cross-platform crash proof, or process-state snapshot.

4 Workspace Execution

Draft.

5 Capture and Publication

Draft.

Stage	Time driver	Live or peak space driver
Session creation	$O(L)$ manifest cloning, path resolution, lower validation, and mount configuration in userspace	$O(L)$ metadata per lease/session and $O(R)$ across live leases; fresh upper/work directories do not eagerly copy the project
Capture	$O(U)$ metadata/path work plus per-directory sorting	$O(U)$ change metadata; regular-file payload remains referenced on disk
Plan/resolve	Changed-path routing and repeated layered fingerprint probes; eligible merge depends on line count and edit distance	Fingerprints and validations scale with $C + F$; merge retains file, line, edit, output, origin, and trace state
Commit	$O(C \log C)$ path aggregation, at least $O(B_p)$ byte hashing/copy/sync, and $O(L)$ manifest work under one writer	The private upper can coexist with staging/final publication data; each unsquashed publish adds a manifest layer
Squash/GC	Manifest/lease planning, layered directory traversal, then a brief serialized commit	Replacement metadata coexists with old layers; leases retain otherwise obsolete history until safe release

Table 1: Operational cost model derived from the v1 source. Logical bytes are not physical allocated bytes; the latter requires filesystem measurements.

6 Lifecycle and Recovery

Draft.

7 Implementation and Operational Interface

7.1 Source-Derived Operational Cost Model

Ephemeral Sandbox does not have one context-free time or space complexity. Each session passes through stages with different scaling variables. Let L be manifest depth, S the number of live sessions, U the number of private-upper entries, C the number of captured changes, F the number of validated source paths, B_p the logical regular-file bytes published, Q the number of concurrent publishers, and $R = \sum_{i=1}^S L_i$ the layer references retained across live leases. Table 1 records source-derived drivers; it is not a performance result.

The durable head transition is serialized by design. In v1, the exclusive writer section includes active-head resolution, eligible text reconciliation, payload hashing and copying, synchronization, promotion, and manifest replacement. Private execution can therefore remain parallel while publication wait grows with Q , B_p , F , or merge work. This is a mechanism-level saturation hypothesis, not evidence that publication is the limiting stage for any workload.

Text reconciliation is bounded to 8 MiB per input, but that byte gate is not an independent memory bound for the line diff. For total line count n , reached edit distance D , and $K = \min(n, 200,000)$, the current implementation retains an $O(KD)$ frontier trace and can approach an $O(K^2)$ trace envelope. The evaluation must stress line count and edit shape in addition to file bytes.

7.2 Operational Interface

Draft pending PW3. This subsection will describe the role-separated management, runtime, and read-only observability clients and the final tagged catalog-derived contract.

8 Evaluation

8.1 RQ3: Latency, Storage, and Resource Scaling

RQ3 tests the source-derived cost model rather than assuming that worker count alone explains scaling. The protocol independently varies worker/publisher count, manifest depth, live-session count and lease age,

private-upper entry count and fan-out, changed and validated paths, published logical bytes, conflict class, and eligible-merge bytes, lines, similarity, and edit distance. Storage cases include small edits to large files, many small files, and incompressible controls. All factors, warmups, repeats, and stopping rules must be fixed before the final source freeze.

For each stage, we report latency distributions, CPU, RSS/PSS, I/O, failures, and exact phase boundaries. Publication additionally reports writer-lock wait and hold time and decomposes planning, resolution/merge, hashing, copy, synchronization, and manifest transition. Storage reports logical, allocated, shared, and exclusive bytes for upper, work, staging, published layers, and lease-retained history. Squash reports plan, build, commit, and garbage collection separately. No cost-model term becomes a paper result until it is reproduced from frozen raw artifacts and analysis code.

The remaining RQ1–RQ5 methodology and all numerical results are pending protocol and evidence lock.

9 Limitations and Related Work

9.1 Cost and Implementation Limits

Workspace isolation removes one source of interference; it does not solve task decomposition, model quality, semantic compatibility, verification, resource ownership, or general agent coordination. v1 also exposes concrete scaling risks. It serializes publication and performs current-head reconciliation plus payload hashing, copying, and synchronization inside the writer critical section. It retains a full manifest per live lease, repeatedly probes layered history during path validation, and can retain obsolete history while long-lived leases protect session views. Its text merge is byte-limited but does not yet have an independently small line/edit-distance trace bound. These are source-derived limitations whose practical severity remains to be measured.

The copy-on-write workspace abstraction is not a claim of constant or negligible physical storage. Kernel copy-up, sparse allocation, compression, extent sharing, page cache, and filesystem metadata are backend-dependent. A small edit to a large lower file can produce a large logical upper file, and v1 publication copies the resulting regular-file payload into a new layer. Accordingly, the evaluation distinguishes logical from allocated, shared, and exclusive bytes.

9.2 LayerStack 2.0 as a Candidate Evolution

LayerStack 2.0 is future work, not part of the v1 contribution. Its migration protocol proposes a qualified single-filesystem storage domain and compares a copy-based v1 layout, a co-located copy control, and a reflink-required candidate. The target is to reduce byte-transfer and allocated-storage amplification without changing image compatibility, blame, squash, remount, active execution, crash, memory, daemon-health, or privilege behavior. A reflink is not $O(1)$: clone work depends on filesystem and extent structure, and later writes allocate changed extents.

The completed Windows feasibility evidence is negative and narrowly scoped. On the pinned stock Windows Docker Desktop/WSL 2 storage cell, direct `FICLONE` returned `errno=95`; the integrated Windows candidate therefore remained blocked. A future design requires capability detection and a correctness-preserving copy fallback, and this paper makes no Windows reflink storage or performance claim.

Other targets follow directly from the v1 model: shorten the writer critical section while preserving all-or-none and crash visibility semantics; share or compact immutable manifest state across leases; index revision-keyed path, fingerprint, and ignore lookups; and reject merge work through independently bounded CPU/memory admission.

9.3 Related Work

Draft pending PW4 and final citation verification.

10 Conclusion

Draft.

References